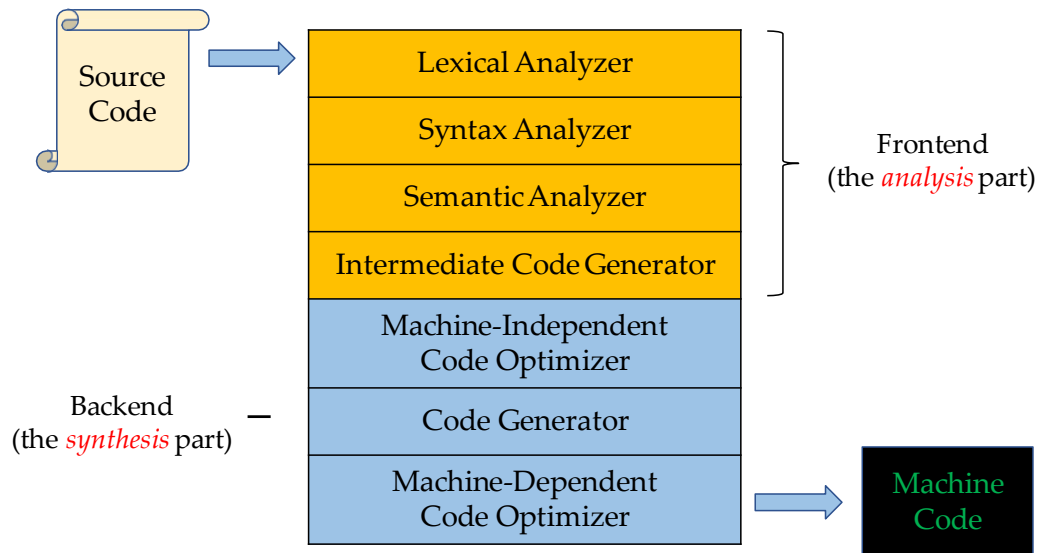


The Structure of a Compiler



编译器的结构

输入：源代码

前端：（分析部分）

词法分析器

语法分析器

语义分析器

中间代码生成器

后端：（合成部分）

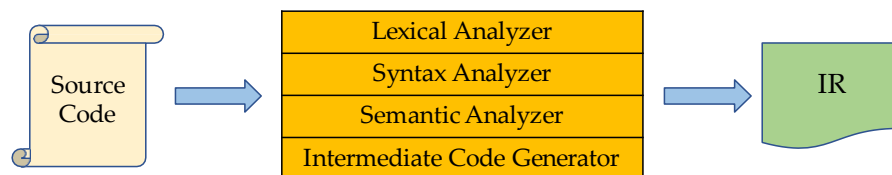
机器无关代码优化器

代码生成器

机器相关代码优化器

输出：机器码

The Frontend (前端) of a Compiler



- Breaks up the source program into **constituent pieces** and imposes a **grammatical structure** on them
- Uses the grammatical structure to create an **intermediate representation (IR)** of the source program
- Collect the information about the source program and stores it in a data structure called **symbol table** (will be passed to backend with IR)

编译器的前端

词法分析器

语法分析器

语义分析器

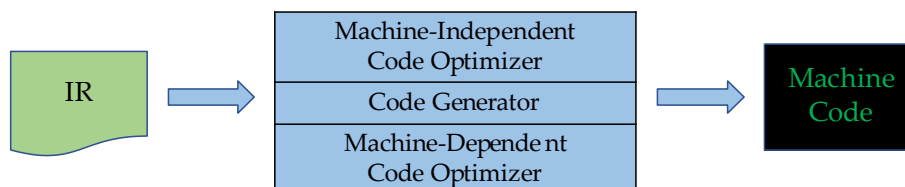
中间代码生成器

将源程序分解为多个组成部分，并在其上强加语法结构

使用语法结构创建源程序的中间表示（IR）

收集有关源程序的信息，并将其存储在称为符号表的数据结构中（将与 IR 一起传给后端）

The Backend (后端) of a Compiler



- Constructs the target program (typically, in machine language) from the IR and the information in the symbol table
- Performs code optimizations during the process

编译器的后端

机器无关代码优化器

代码生成器

机器相关代码优化器

根据 IR 和符号表中的信息构建目标程序（通常使用机器语言）

在此过程中执行代码优化

Lexical Analysis (Scanning, 词法分析)

Character stream $\longrightarrow$ **Lexical Analyzer** $\longrightarrow$ Tokenstream

- The lexical analyzer (lexer/tokenizer/scanner) breaks down the source code into a sequence of “lexemes” (词素) or “words”
- For each lexeme, produce a “token” (词法单元) in the form:

$\langle$ token-name, attribute-value $\rangle$

An **abstract symbol** that is used during syntax analysis

Points to an entry in the symbol table.
Info in the table entry is for semantic analysis and code generation.

字符流 $\rightarrow$ 词法分析 $\rightarrow$ token 流

词法分析器将源代码分解为一系列 词素 或 “words”

对于每个词素，生成一个形式上为 $\langle$ token 名, 属性值 $\rangle$ 的词法单元

词法单元中单元名称：语法分析过程中使用的抽象符号

属性值：指向符号表中的一个条目。表条目中的信息用于语义分析和代码生成。

Lexemes vs. Tokens

- A **lexeme** is a string of characters that is a lowest-level syntactic unit in the programming language
 - "words" and punctuation of the programming language (**instance**)
- A **token** is a syntactic category representing a class of lexemes
 - **In English:** Noun, Verb, Adjective...
 - **In programming language:** Identifier, Keyword, Whitespace... (**pattern**)

<https://courses.cs.vt.edu/~cs1104/Compilers/Compilers.070.html>

词素 vs token

词素是一个字符串，是程序设计语言中最低级别的句法单元

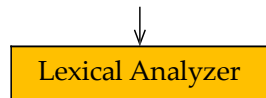
- “单词”和程序设计语言的标点符号记号

token 是一种语法类别，代表一类词汇

- 英语：名词，动词，形容词...
- 在编程语言中：标识符，关键字，空白... (模式)

Lexical Analysis (Example)

position = initial + rate * 60



<id, 1>

<=>

<id, 2>

<+>

<id, 3>

<*>

<60>

SYMBOL TABLE

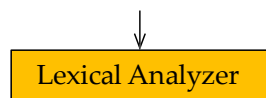
1	position	...
2	initial	...
3	rate	...

Note: <=>, <+>, <*>, <60> are not in the defined form. This is for notational convenience. <=> could have been <assign, -> and <60> could have been <number, 4>.

注意: <=>, <+>, <*>, <60>不是定义的形式。这是为了便于注释。: <=>可能是 <assign, -> and <60>可能是<number, 4>.

Lexical Analysis (Analogy)

position = initial + rate * 60



<id, 1>

<=>

<id, 2>

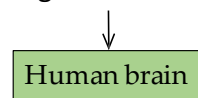
<+>

<id, 3>

<*>

<60>

BUPT is a good university



<noun, "BUPT">

<verb, "is">

<article, "a">

<adjective, "good">

<noun, "university">

Syntax Analysis (Parsing, 语法分析)

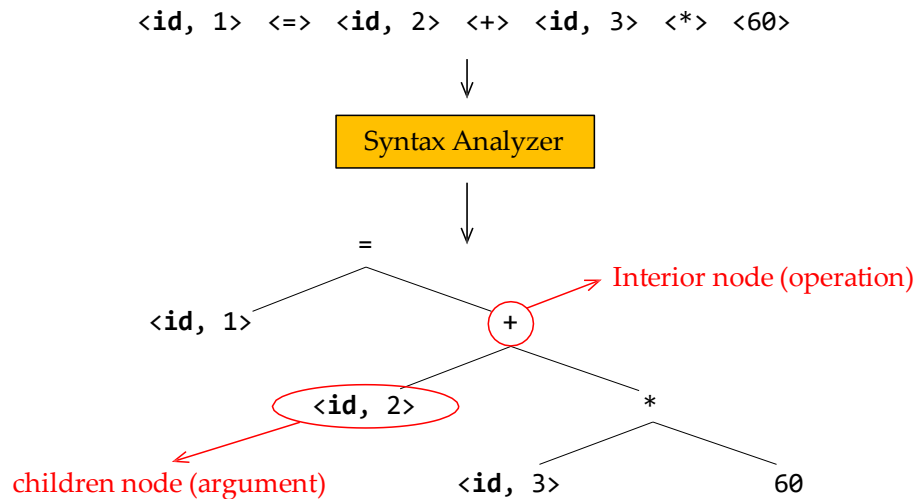


- The syntax analyzer (parser) uses the **token names** produced by the lexer to create an intermediate representation that depicts the grammar structure of the token stream, typically a *syntax tree*
- Each interior node represents an **operation** and the children of the node represent the **arguments** of the operation

语法分析器（解析器）使用 lexer 生成的 token 的名称来创建一个中间表示，该中间表示描述 token 流的语法结构，通常用语法树来表示

每个内部节点表示一个操作，内部节点的子节点表示该操作的参数

Syntax Analysis (Example)

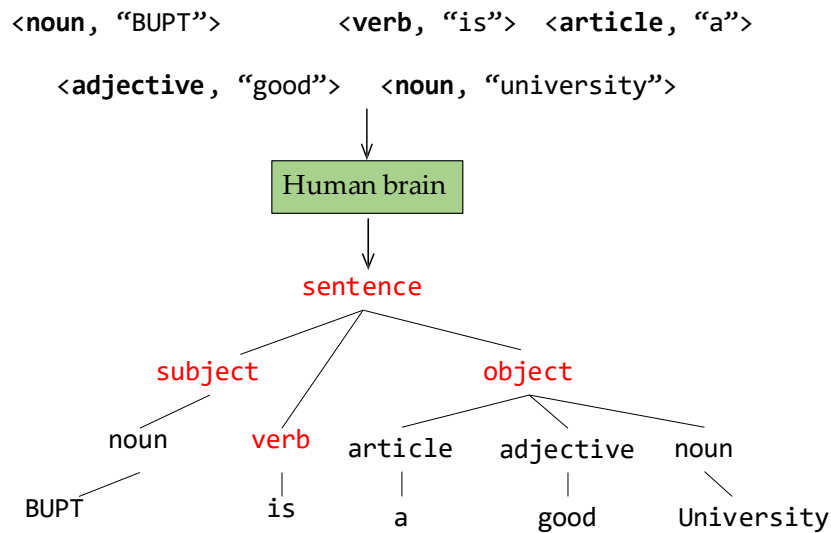


Fall 2024

BUPT Compilers

26

Syntax Analysis in English



Fall 2024

BUPT Compilers

27

BUPT 为名词, is 为动词, a 为冠词, good 为形容词, university 为名词

Semantic Analysis (语义分析)



- The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for **semantic consistency** with the language definition
- Also gathers **type information** for type checking, type conversion, and intermediate code generation

语义分析器使用语法树和符号表中的信息来检查源程序与语言定义的语义一致性
还收集用于类型检查、类型转换和中间代码生成的类型信息

What is Semantics?

- The **syntax** of a programming language describes the **proper form** of its programs
- The **semantics** of a programming language describes the **meaning** of its programs, i.e., what each program does when it executes

什么是语义?

编程语言的语法描述程序的正确形式

编程语言的语义描述程序的含义，即每个程序在执行时做什么

Semantic Analysis in English

Jack said Jerry left his assignment at home.

What does “his” refer to? Jack’s or Jerry’s?

Jack said Jack left his assignment at home.

How many Jacks? Which one left the assignment?

Examples are from Aiken’s notes (Stanford CS143)

英语中的语义分析

杰克说杰瑞把他的作业落在家里了。“他的”指的是什么？杰克的还是杰瑞的？

杰克说杰克把他的作业落在家里了。有多少个杰克？哪一个把作业落在家里了？

Semantic Analysis in Programming

- Understanding the meaning of a program is very hard 😞
- Compilers perform only very limited analysis to catch semantic inconsistencies.

```
1. {  
2.   int Jack = 3;  
3.   {  
4.     int Jack = 4;  
5.     print Jack;  
6.   }  
7. }
```

Which value will be printed?

Programming languages define strict rules to avoid ambiguities.

Compiler will bind Jack at line 5 to its inner definition at line 4.

程序中的语义分析

理解程序的含义是非常困难的。

编译器只执行非常有限的分析来捕捉语义不一致。

编程语言定义了严格的规则来避免歧义。

编译器将把第 5 行的 Jack 绑定到第 4 行的内部定义。

Type Checking (类型检查)

- An important part of semantic analysis is type checking
- Compilers check that each operator has matching operands (of correct types)

Example: Many language definitions require an array index to be an integer.

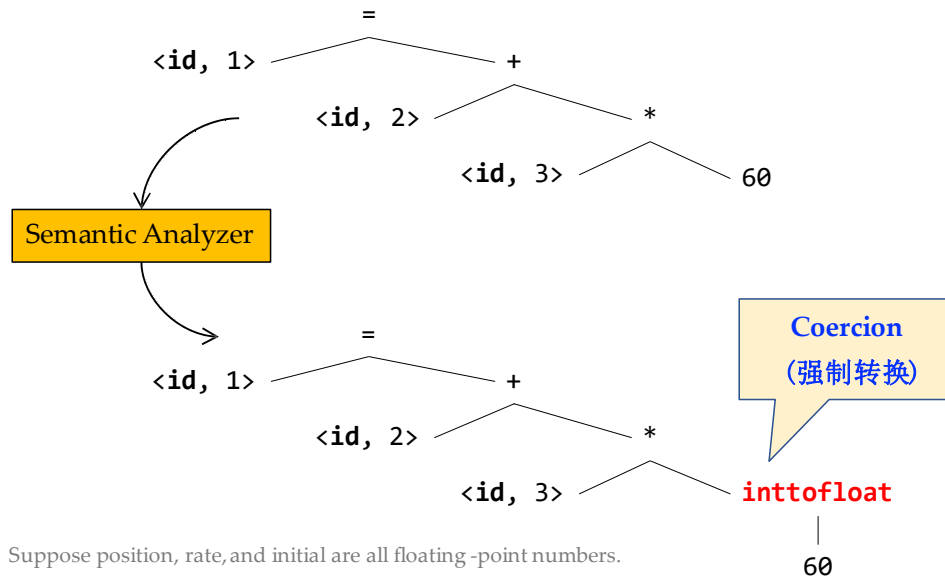
Compilers should report an error if the argument of an array element access operation is a floating-point number.

类型检查是语义分析的一个重要部分

编译器检查每个运算符是否具有匹配的操作数（正确类型）

示例：许多语言定义要求数组索引为整数。如果数组元素访问操作的参数是浮点数，则编译器应报告错误。

Semantic Analysis (Example)



Intermediate Code Generation

(中间代码生成)

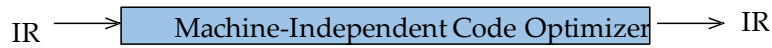


- After semantic analysis, compilers generate an intermediate representation, typically *three-address code* (三地址码)
 - *Assembly-like instructions* with three operands per instruction
 - Each operand acts like a register
 - Each assignment instruction has at most one operator on the RHS
 - Easy to translate into machine instructions of the target machine

语义分析后，编译器生成一个中间表示，通常为三地址码

- 每个指令有三个操作数的类汇编指令
- 每个操作数的作用就像一个寄存器
- 每个赋值指令在 RHS 上最多有一个运算符
- 易于翻译成目标机器的机器指令

Machine-Independent Code Optimization (机器无关的代码优化)



- Akin to article editing/revising in English
- Improve the intermediate code for better target code
 - Run faster
 - Use less memory
 - Shorter code
 - Consume less power ...

使用英语编辑/修改文章改进中间代码

以获得更好的目标代码

- 跑得更快
- 使用更少的内存
- 较短代码
- 耗电更少...

Code Optimization (Example)

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

1. **60 is a constant integer value.** Its conversion to floating-point can be done once and for all at compile time
2. **t3** is only used for value transmitting

Optimization

```
t1 = id3 * 60.0
id1 = id2 + t1
```

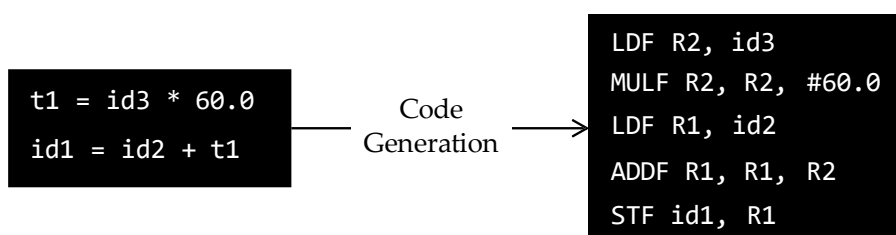
60 是一个恒定的整数值。它到浮点的转换可以在编译时一次性完成。

t3 仅用于值传输

Code Generation (代码生成)

IR $\longrightarrow$ Code Generator $\longrightarrow$ Target-machine code

- Map IR to target language, analogous to human translation
- It is crucial to **allocate register and memory** to hold values



将 IR 映射到目标语言，类似于人工翻译

分配寄存器和内存以保存值至关重要

Symbol Table Management

- **Performed by the frontend**, symbol table is passed along with the intermediate code to the backend
- Record the **variable names** and various **attributes**
 - storage allocated, type, scope
- Record the **procedure names** and various **attributes**
 - the number and type of arguments
 - the way of passing arguments (by value or by reference)
 - the return type

符号表管理

由前端执行，符号表与中间代码一起传递到后端

记录变量名称和各种属性

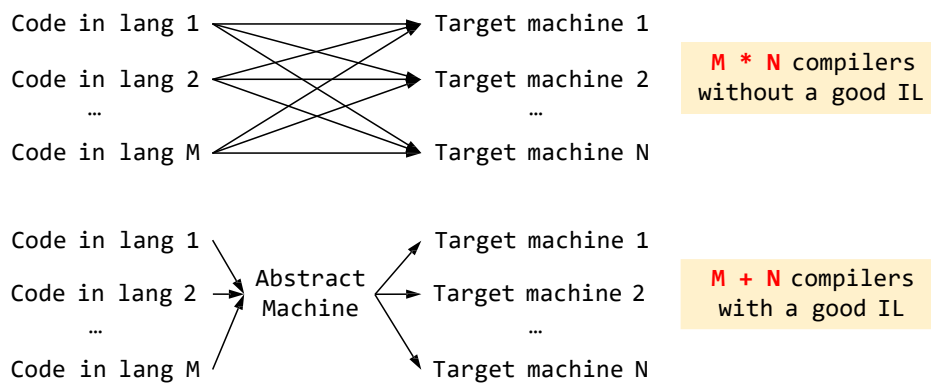
- 分配的存储、类型、范围

记录过程名称和各种属性

- 参数的数量和类型
- 传递参数的方式（通过值或引用）
- 返回类型

Intermediate Language (IL)

- Intermediate code is in IL (e.g., three-address code)
- A good IL eases compiler implementation



中间语言 (IL)

中间代码在 IL 中 (例如, 三个地址代码)

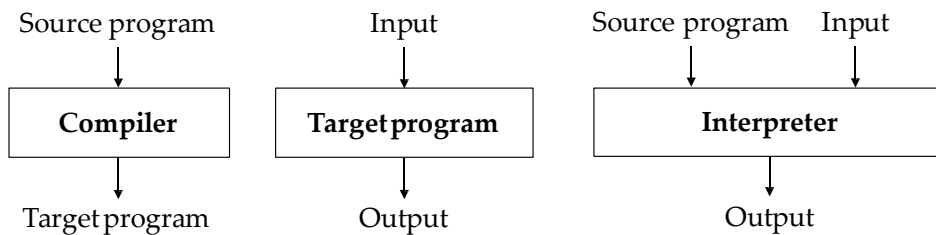
一个好的 IL 简化了编译器的实现

Compilers vs. Interpreters

1

A **compiler** translates **source programs** written in high-level languages into **machine codes** that can run directly on the target computer.

An **interpreter** **directly executes** each statement in the source code, without requiring the program to have been compiled into machine codes.



编译器将用高级语言编写的源程序翻译成可以直接在目标计算机上运行的机器代码。

解释器直接执行源代码中的每个语句，而不需要将程序编译成机器代码。

Compilers vs. Interpreters

2

Interpreters often take less time to analyze the source code: they simply parse each statement and execute it (e.g., Python code).

In comparison, compilers typically analyze the relationships among statements (e.g., control and data flows) to enable optimizations.

3

Interpreters continue executing a program until the first error is met, in which case they stop.

For compiled languages, programs are executable only after they are successfully compiled.

解释器通常花更少的时间来分析源代码：他们只需解析每个语句并执行它（例如 Python 代码）。

相比之下，编译器通常会分析语句（例如，控制流和数据流）之间的关系，以实现优化。

解释器持续执行程序，直到遇到第一个错误才停止。

对于编译语言，程序只有在成功编译后才能执行。